Technical Report: Preliminary Investigation of the Shimmer Sensor Dataset

Project: Machine Learning Fundamentals and Transfer Learning

Organization: Centre de Développement des Technologies Avancées (CDTA)

Dataset: Shimmer sensor dataset

Objective: Investigate dataset structure, timestamp continuity, potential recording boundaries, sensor values, and 128-sample segmentation before developing the machine-learning pipeline.

---

1. Introduction

This report documents the preliminary investigation of the Shimmer dataset provided for the CDTA internship.

The investigation was conducted using Python scripts executed through the Windows Command Prompt. The tests examined:

- Dataset structure and sample counts.
- Timestamp intervals and discontinuities.
- Potential recording boundaries.
- Accelerometer measurements around detected boundaries.
- The suitability of 128-sample windows.
- The alignment of timestamp boundaries with 128-sample blocks.

The purpose is to understand the data before implementing feature extraction, comparing classical machine-learning algorithms, and preparing the transfer-learning experiment.

2. Dataset Structure

The dataset contains seven MATLAB files:

- `Fall.mat`
- `Jump.mat`
- `Lay.mat`
- `Run.mat`
- `Sit.mat`
- `Stand.mat`
- `Walk.mat`

Each file contains a MATLAB variable named `data`, represented as a matrix with 11 columns. The preliminary interpretation of the columns is:

Column index	Preliminary interpretation
0	Timestamp
1	Accelerometer X
2	Accelerometer Y

Column index	Preliminary interpretation
3	Accelerometer Z
4	Gyroscope X
5	Gyroscope Y
6	Gyroscope Z
7–10	Additional sensor channels

Note: This column mapping is provisional. The column names, units, and exact meanings should be confirmed using the dataset documentation or the supervisor's instructions.

2.1 Number of samples

The following sample counts were obtained during the initial investigation.

Activity	Number of samples
Fall	6,016
Jump	18,304
Lay	42,240
Run	36,992
Sit	44,032
Stand	52,480
Walk	45,696
Total	**245,760**

The dataset contains **245,760 samples across seven activity classes**.

The number of samples differs considerably between classes. Stand contains the most samples, with 52,480, while Fall contains the fewest, with 6,016.

These differences should be considered when evaluating class balance. However, raw sample counts do not necessarily represent the number of independent recordings or independent training examples.

3. Timestamp Analysis

3.1 Median timestamp interval

The first timestamp investigation calculated the differences between consecutive timestamps:

$$\Delta t_i = t_{i+1} - t_i$$

The median positive timestamp interval was:

19.53125

This value was observed in all seven files.

If the timestamp unit is milliseconds, the corresponding approximate sampling frequency is:

$$f_s = 1000 / 19.53125 \approx 51.2 \text{ Hz}$$

However, the timestamp units have not yet been independently confirmed.

The supervisor specified that 128 samples correspond to 2.56 seconds, implying:

$$f_s = 128 / 2.56 = 50 \text{ Hz}$$

The estimated frequency based on the timestamps and the supervisor's specified frequency are close but not identical.

The timestamp units and sampling frequency should therefore be confirmed before finalizing the preprocessing methodology. The window size should remain 128 samples unless the supervisor advises otherwise.

3.2 Nonpositive timestamp intervals

The initial timestamp test identified the following nonpositive intervals:

Activity	Nonpositive intervals
Fall	12
Jump	16
Lay	11
Run	12
Sit	9
Stand	15
Walk	12
Total	**87**

A nonpositive interval occurs when a timestamp is equal to or smaller than the preceding timestamp.

These observations show that the timestamps are not strictly increasing throughout the complete files.

Possible explanations include timestamp resets, concatenated recordings, or characteristics of the data-export process. The underlying cause has not yet been established.

3.3 Large timestamp gaps

The initial investigation also identified large timestamp changes.

Activity	Large gaps
Fall	36
Jump	65
Lay	44
Run	37
Sit	32
Stand	58
Walk	39
Total	**311**

These gaps were identified using the threshold implemented in the initial timestamp-checking script.

A large timestamp change can indicate a discontinuity, but it does not automatically prove that the sensor recording stopped or that a new recording began.

3.4 Initial boundary findings

The initial investigation reported 398 candidate boundaries when nonpositive intervals and large timestamp gaps were combined.

These were preliminary counts based on the original detection procedure.

A later script, `check_block_boundaries.py`, used a specific rule to identify timestamp boundaries and examine whether they aligned with 128-sample blocks. Its results are presented in Section 6.

The two sets of counts should not be treated as directly interchangeable because the scripts used different boundary-detection rules.

4. Inspection of Timestamp Discontinuities

A second investigation examined timestamps immediately before and after selected boundaries.

4.1 Boundaries aligned with 128-sample blocks

Many detected timestamp discontinuities occurred after samples 127, 255, 383, 639, 767, and other indices corresponding to the ends of consecutive 128-sample blocks.

This pattern was observed across several activity files.

It is consistent with the supervisor's specified window size of 128 samples.

However, alignment does not prove that the data was originally recorded as independent 128-sample windows. It may reflect the original segmentation, concatenation, or export process.

4.2 Irregular boundaries

Other discontinuities occurred at positions that were not aligned with the expected 128-sample boundaries.

Examples identified during the earlier investigation included:

- `Lay.mat`: sample index 5424.
- `Run.mat`: sample index 557.
- `Stand.mat`: sample index 1779.
- `Walk.mat`: sample index 17409.

These positions require attention because a discontinuity inside a window could cause samples from different recording segments to be combined.

Whether that combination is invalid depends on the underlying meaning of the timestamp change.

4.3 Examples of timestamp jumps and resets

The boundary inspection revealed substantial timestamp changes.

Examples from `Fall.mat` included:

- Between sample indices 127 and 128, the timestamp increased by approximately 21,230.469.
- Between sample indices 255 and 256, the timestamp decreased by approximately 90,878.906.

Examples from `Jump.mat` included:

- Between sample indices 383 and 384, the timestamp increased by approximately 4,722,968.75.
- Between sample indices 639 and 640, the timestamp decreased by approximately 4,698,300.781.

These results indicate that timestamp progression is not continuous throughout each file.

The observed changes may be associated with recording boundaries or timestamp resets, but their exact cause remains unknown.

5. Accelerometer Inspection Around Boundaries

A third investigation examined accelerometer measurements immediately before and after selected timestamp boundaries.

5.1 Sensor values

The examined accelerometer measurements were numeric on both sides of the selected boundaries.

For example, in `Fall.mat`, the acceleration vectors around the boundary between samples 127 and 128 were approximately:

$$\mathbf{a}_{127} = [-0.3646, 7.9393, 5.5964]$$

$$\mathbf{a}_{128} = [-0.2489, 9.5483, 0.0898]$$

The Euclidean distance between these vectors was approximately:

$$\|\mathbf{a}_{128} - \mathbf{a}_{127}\| \approx 5.738$$

Other examined boundaries showed different magnitudes of acceleration change. Some changes in Fall.mat exceeded 13, while some examined boundaries in Jump.mat exceeded 20.

5.2 Interpretation

A sudden acceleration change does not necessarily indicate corrupted data.

Activities such as jumping and falling can naturally produce rapid changes in acceleration. Sensor orientation can also influence the values recorded on each axis.

Likewise, numeric sensor values do not guarantee that samples on opposite sides of a timestamp boundary belong to the same recording.

Consequently, the accelerometer values alone cannot determine whether a boundary represents a genuine recording transition.

The sensor values should not be automatically deleted because of timestamp resets or large differences in acceleration.

6. Detailed Timestamp Boundary Alignment Test

6.1 Objective

The script `check_block_boundaries.py` was used to examine all seven activity files.

The purpose was to determine:

1 The number of timestamp boundaries identified by the script.

2 How many boundaries occurred at the end of a 128-sample block.

3 How many boundaries occurred inside a 128-sample block.

The script considered a boundary to be either:

- A nonpositive timestamp interval, or

- A timestamp change whose absolute magnitude exceeded five times the median positive timestamp interval.

This detection rule is important when interpreting the results.

6.2 Results for all seven files

The script produced the following results:

Activity	Total samples	Timestamp boundaries	Aligned with 128-sample blocks	Inside 128-sample blocks
Fall	6,016	45	45	0
Jump	18,304	76	70	6
Lay	42,240	44	40	4
Run	36,992	41	39	2
Sit	44,032	30	30	0
Stand	52,480	61	59	2
Walk	45,696	41	36	5
Total	**245,760**	**338**	**319**	**19**

According to this script's detection rule, 338 timestamp boundaries were identified. Of these:

- **319 boundaries were aligned with 128-sample block boundaries.**
- **19 boundaries occurred inside 128-sample blocks.**

Approximately 94.4% of the detected boundaries were aligned with the specified window boundaries, while approximately 5.6% occurred inside blocks.

These percentages describe the alignment of the boundaries detected by this particular script. They are not estimates of the proportion of corrupted samples or invalid windows.

6.3 Interpretation by activity

Fall

All 45 detected boundaries were aligned with 128-sample block boundaries. No internal boundaries were identified by this script.

This is consistent with regular block alignment, although it does not establish the original recording structure.

Jump

The script identified 76 boundaries. Of these, 70 were aligned with block boundaries and six occurred inside blocks.

The six internal boundaries were consecutive timestamp intervals between samples 14713–14714 and 14718–14719. Each reported timestamp change was zero.

These observations indicate a repeated timestamp value over several consecutive intervals.

The sensor measurements around this region should be inspected before deciding how to handle it.

Lay

The script identified 44 boundaries. Forty were aligned with block boundaries, while four occurred inside blocks.

The reported internal timestamp changes were:

Samples	Timestamp change
23766–23767	800.781
23919–23920	683.594
24061–24062	527.344
40390–40391	488.281

These positive changes exceeded the script's detection threshold.

They should be investigated to determine whether they represent recording gaps or another characteristic of the data.

Run

The script identified 41 boundaries. Thirty-nine were aligned with block boundaries, while two occurred inside blocks.

Samples	Timestamp change
27121–27122	273.438
27810–27811	253.906

Both changes were positive but large enough to meet the detection rule.

Sit

All 30 detected boundaries were aligned with 128-sample block boundaries.

No internal boundaries were identified by this script.

Stand

The script identified 61 boundaries. Fifty-nine were aligned with block boundaries, while two occurred inside blocks.

Samples	Timestamp change
19420–19421	1054.688
35610–35611	136.719

These boundaries require further interpretation in the context of the original recording process.

Walk

The script identified 41 boundaries. Thirty-six were aligned with block boundaries, while five occurred inside blocks.

Samples	Timestamp change
19344–19345	156.250
19729–19730	1796.875
35371–35372	1230.469
45124–45125	605.469
45676–45677	488.281

These results show that internal timestamp discontinuities are not limited to one activity class.

6.4 Why the boundary counts differ from the earlier results

The earlier investigation reported 398 candidate boundaries. The later script identified 338.

This difference does not necessarily indicate an error in either test.

The scripts used different detection procedures and thresholds. The earlier test counted nonpositive intervals and large gaps according to its original criteria, while `check_block_boundaries.py` used the absolute timestamp-change threshold of five times the median positive interval, together with nonpositive intervals.

Therefore, the counts represent different definitions of a candidate boundary.

For reproducibility, the final preprocessing pipeline should use one clearly documented boundary-detection rule, with the threshold justified according to the timestamp units and expected sampling behavior.

7. Investigation of 128-Sample Blocks

The script `inspect_blocks.py` examined the first five blocks of Fall.mat, Jump.mat, and Walk.mat.

7.1 Number of complete blocks

Activity	Total samples	Complete 128-sample blocks	Remaining samples
Fall	6,016	47	0
Jump	18,304	143	0
Walk	45,696	357	0

All three files contain exact multiples of 128 samples.

The remaining four files also have exact multiples of 128 samples:

Activity	Total samples	Complete 128-sample blocks
Lay	42,240	330
Run	36,992	289
Sit	44,032	344
Stand	52,480	410

Consequently, all seven files can be mathematically divided into complete, non-overlapping 128-sample blocks without leftover samples.

However, mathematical divisibility does not establish whether all such blocks are valid independent windows.

7.2 Timestamp continuity within examined blocks

For the first five blocks of Fall, Jump, and Walk:

- No nonpositive timestamp intervals were detected inside the examined blocks.
- The median positive interval was 19.53125.

These findings indicate consistent timestamp progression within the examined portions of the files.

However, the test covered only the first five blocks of three files. It did not verify every window in the complete dataset.

The later boundary-alignment test identified internal timestamp discontinuities elsewhere in the files.

7.3 Acceleration statistics

The mean and standard deviation of the three accelerometer axes were calculated for each examined block.

For example, the mean acceleration values in the first five Walk blocks were:

Block	Mean X	Mean Y	Mean Z
1	0.901	9.787	-0.444
2	0.484	9.559	-0.414
3	0.702	9.779	-0.140
4	0.311	9.568	-0.382
5	0.496	9.716	-0.188

The mean Y-axis values were relatively similar across these five windows.

The first Fall block had standard deviations of approximately:

[3.518, 9.195, 7.379]

The first Jump block had standard deviations of approximately:

[2.474, 7.205, 6.824]

These statistics demonstrate that the sensor signals have measurable variation within the windows.

However, the results do not establish classification performance or prove that the current segmentation is correct.

8. Summary of Findings

The investigations established the following:

1. The dataset contains seven activity files and 245,760 samples.
2. Each file contains an 11-column matrix.
3. The median positive timestamp interval is 19.53125 across all seven files.
4. The timestamps are not strictly increasing throughout the complete files.
5. The initial timestamp test identified 87 nonpositive intervals and 311 large gaps under its detection criteria.
6. Many timestamp discontinuities occur at the ends of 128-sample blocks.
7. The later boundary-alignment script identified 338 boundaries using its own detection rule.
8. Of those 338 boundaries, 319 were aligned with 128-sample blocks and 19 occurred inside blocks.
9. Jump, Lay, Run, Stand, and Walk contain internal timestamp boundaries under the later script's criteria.
10. The examined accelerometer values were numeric on both sides of the selected boundaries.
11. All seven files contain exact multiples of 128 samples.

12 The first five blocks of Fall, Jump, and Walk had internally increasing timestamps.

13 The timestamp units and exact column definitions remain unconfirmed.

14 The cause of the timestamp discontinuities has not yet been established.

The most important unresolved issue is whether the timestamp discontinuities correspond to actual recording boundaries or are simply characteristics of the exported data.

9. Implications for the Machine-Learning Pipeline

9.1 Segmentation

The supervisor specified windows of 128 samples, corresponding to 2.56 seconds.

This requirement should be preserved unless the supervisor advises otherwise.

However, the 19 internal boundaries identified by the later script require attention. If any of them represent genuine recording transitions, a window that spans such a boundary could combine unrelated segments.

The correct approach is therefore to determine what these discontinuities represent before finalizing segmentation.

9.2 Feature extraction

The supervisor instructed that classical machine-learning features should be extracted for the X, Y, and Z accelerometer axes.

The initial feature set can include:

- Mean.
- Standard deviation.
- Minimum.
- Maximum.

Additional features, such as range, interquartile range, root mean square, and magnitude statistics, may be considered if appropriate.

Each valid window should be represented by a consistent feature vector and its corresponding activity label.

9.3 Acceleration magnitude

For the transfer-learning experiment, the supervisor specified the magnitude signal:

$$\text{Mag} = \sqrt{(X^2 + Y^2 + Z^2)}$$

This transforms the three accelerometer axes into a single magnitude signal.

The exact input representation and transfer-learning architecture should be determined in the next stage of the project.

9.4 Data quality

The current tests do not constitute a complete data-quality assessment.

The next assessment should examine all relevant sensor columns for:

- Missing values.

- Non-numeric values.
- Infinite values.
- Unexpected data types.
- Timestamp discontinuities.
- Potential recording boundaries.

Samples should not be deleted solely because timestamps reset or acceleration changes sharply. Any exclusions should be supported by a documented rule.

9.5 Evaluation methodology

Once the dataset has been segmented and the features extracted, two or three classical models can be compared, following the supervisor's instructions.

Possible models include KNN, SVM, and Decision Tree.

Evaluation should include accuracy, balanced accuracy, precision, recall, F1-score, and confusion matrices.

If multiple recordings or participants are present, the train/test split should avoid placing closely related windows from the same recording in both sets whenever possible. Otherwise, performance estimates may be overly optimistic.

10. Recommended Next Steps

Step 1: Confirm the data structure with the supervisor

Ask whether the `.mat` files contain:

- Multiple recordings concatenated together.
- Previously segmented windows.
- Timestamp resets caused by recording or export procedures.
- A known explanation for boundaries occurring at multiples of 128 samples.

Step 2: Confirm timestamp units and sampling frequency

Clarify the timestamp units and the expected sampling frequency, particularly because the observed median interval implies approximately 51.2 Hz if the units are milliseconds, while the supervisor's window specification implies 50 Hz.

Step 3: Investigate the 19 internal boundaries

Inspect the sensor values and timestamps around these positions.

Determine whether they represent actual recording transitions, repeated timestamps, brief gaps, or another phenomenon.

Do not automatically discard the affected samples.

Step 4: Finalize segmentation

Once the meaning of the boundaries is understood, implement the 128-sample windowing procedure.

If genuine recording boundaries are identified, ensure that windows do not cross them.

Step 5: Complete data-quality assessment

Check all relevant columns, document invalid values, and define a consistent cleaning procedure.

Step 6: Extract features

Calculate the required statistics for the three accelerometer axes within each valid window.

Preserve the activity labels and useful metadata, such as source file and recording or segment identifiers when available.

Step 7: Compare classical models

Evaluate KNN, SVM, and Decision Tree using a documented experimental protocol and appropriate metrics.

Step 8: Prepare transfer learning

Calculate the acceleration magnitude and prepare the input representation according to the supervisor's instructions.

Step 9: Document the experiments

Record preprocessing decisions, segmentation rules, feature definitions, model configurations, evaluation methods, and results.

11. Limitations

This report summarizes preliminary exploratory tests rather than a completed dataset validation.

The remaining limitations include:

- Timestamp units have not been confirmed.
- The full column mapping and sensor units have not been verified against official documentation.
- The causes of timestamp discontinuities remain unknown.
- The sensor-boundary inspection covered selected positions rather than every boundary.
- The first five blocks were examined only for Fall, Jump, and Walk.
- A complete missing-value and invalid-value assessment has not been completed.
- The final segmentation procedure has not yet been validated.
- No model performance results are available from these exploratory tests.

These limitations should be resolved or documented before drawing conclusions from the machine-learning experiments.

12. Conclusion

The preliminary investigation established that the Shimmer dataset contains seven activity files with a total of 245,760 samples. All seven files can be divided mathematically into complete 128-sample blocks, consistent with the window size specified by the supervisor.

Timestamp analysis revealed nonpositive intervals, large timestamp gaps, and discontinuities at both aligned and irregular positions. The later boundary-alignment script identified 338 candidate boundaries, of which 319 were aligned with 128-sample blocks and 19 occurred inside blocks.

The results suggest that the relationship between the timestamp discontinuities and the dataset's recording structure must be clarified before finalizing segmentation.

The accelerometer inspection confirmed that the examined measurements were numeric, but the presence of numeric values does not establish whether adjacent samples belong to the same recording. Similarly, sudden acceleration changes alone are not sufficient grounds for removing samples.

The immediate priority is to confirm the meaning of the timestamp boundaries with the supervisor. Once this is understood, the preprocessing pipeline can be finalized, followed by feature extraction, classical machine-learning comparisons, and the transfer-learning experiment.

This report provides a preliminary technical foundation for developing a reliable and reproducible machine-learning workflow for the CDTA internship.